PYTHON PROGRAMMMING ASSESSMENT TOOL-1

R.Swethika

1921260261

TASK-1

```python
RATE_PER_FAT = 5
SNF_BONUS = 1
DAYS = 10

farmer_id = input("Enter Farmer ID: ")
litres = float(input("Enter milk quantity in litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
shift = input("Enter shift (Morning/Evening): ")

# Validation
if litres < 0 or not 2 <= fat <= 7 or snf < 0:
    print("Invalid input")
else:
    rate = (fat * RATE_PER_FAT) + (snf * SNF_BONUS)
    payment = litres * rate
    total = payment * DAYS

    print("\n--- Milk Payment Details ---")
    print("Farmer ID:", farmer_id)
    print("Shift:", shift)
    print("Rate per litre: Rs.", round(rate, 2))
    print("Payment amount: Rs.", round(payment, 2))
    print("Ten-day total: Rs.", round(total, 2))
```

```
Enter Farmer ID: f101
Enter milk quantity in litres: 20
Enter fat percentage: 4
Enter SNF percentage: 8
Enter shift (Morning/Evening): morning

--- Milk Payment Details ---
Farmer ID: f101
Shift: morning
Rate per litre: Rs. 28.0
Payment amount: Rs. 560.0
Ten-day total: Rs. 5600.0
```

This Python program functions as an automated milk billing and payment receipt system designed for dairy collection centers. It begins by establishing baseline constant values: a rate multiplier of ₹5 for each unit of fat, a bonus of ₹1 for each unit of Solid-Not-Fat (SNF), and a 10-day billing cycle duration. The program then prompts the operator to input specific details for a milk delivery, including the farmer's ID, total quantity in litres, fat percentage, SNF percentage, and the collection shift (morning or evening). Before performing any calculations, a validation check runs to ensure the entered values are logical—making sure the milk quantity and SNF are not negative and that the fat percentage falls within a valid dairy range of 2% to 7%. If any input violates these rules, the program halts and displays an error message. Otherwise, it calculates the price per litre

using the formula , multiplies that rate by the total litres to find the single-shift payout, and scales it across the 10-day period to determine the cumulative earnings. Finally, the program outputs a clean, professional summary receipt showing the farmer's ID, shift, calculated rate per litre, single-shift payment, and the final 10-day total with all financial amounts rounded to two decimal places.

TASK-2

```
print("1. Data Types")
print(type(10))
print(type(10.0))
print(type("10"))
print(type(True))
print(type([10, 8.5]))

# 2. Arithmetic operations
print("\n2. Arithmetic Operations")
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(-7 // 2)

# 3. Floating-point precision
print("\n3. Floating Point Precision")
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)

# 4. Money calculation using float
print("\n4. Money Calculation")
amount = 0.1 + 0.2
print("Using float:", amount)

# Using Decimal
print("Using Decimal:", Decimal("0.1") + Decimal("0.2"))

# Change Decimal(amount) to Decimal(float(amount))
print("Using Decimal(float):", Decimal(float(amount)))

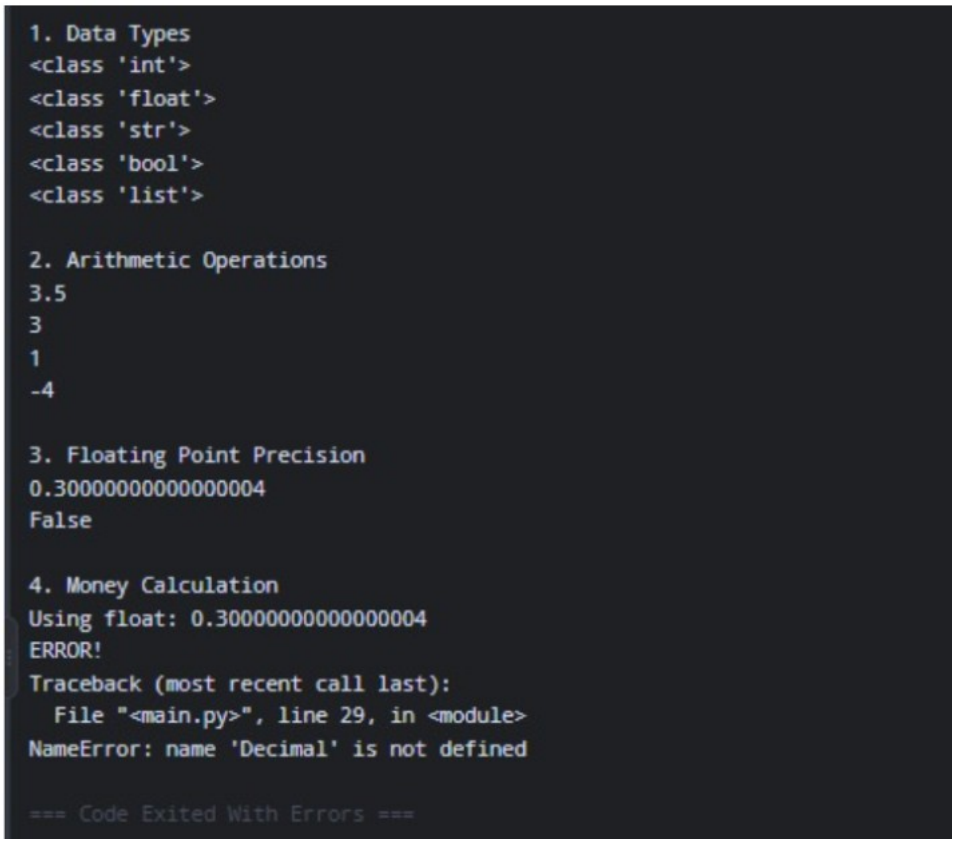
# 5. Compare three methods
print("\n5. Comparing Money Methods")

# Method 1: Decimal
a = Decimal("0.10")
b = Decimal("0.20")
decimal_total = a + b
print("Decimal total:", decimal_total)
```

```
a_paise = 10
b_paise = 20
paise_total = a_paise + b_paise
print("Integer paise total:", paise_total, "paise")
print("Rupees:", paise_total / 100)

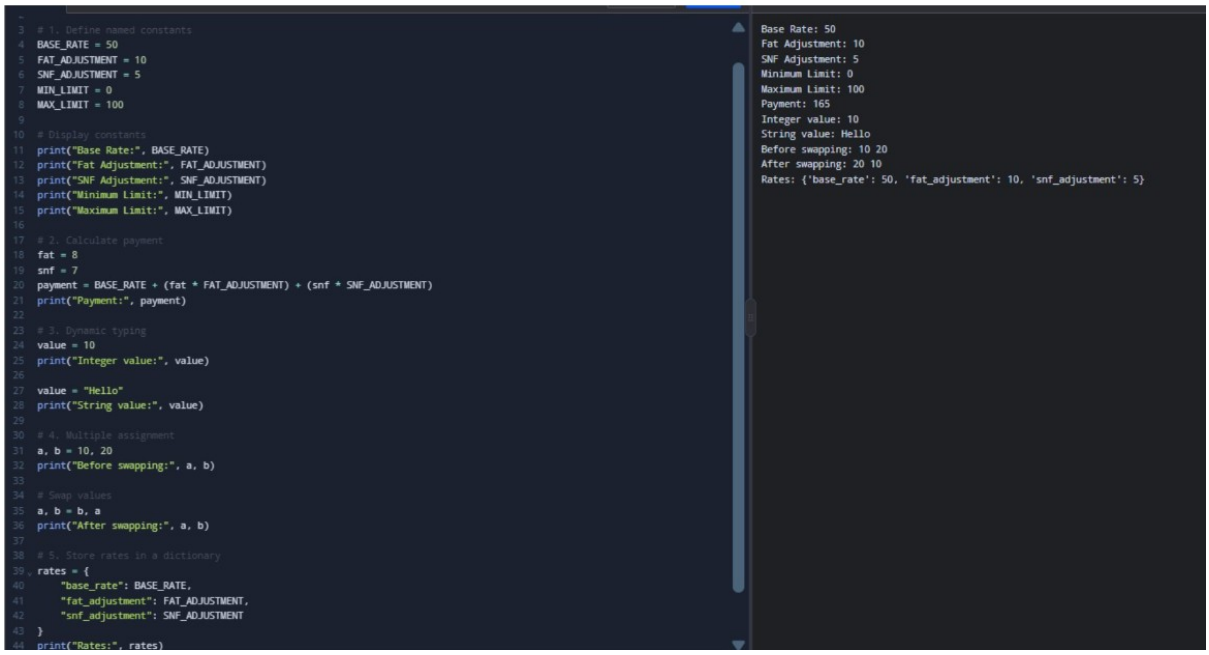
# Method 3: Float with rounding
a = 0.10
b = 0.20
float_total = round(a + b, 2)
print("Float total:", float_total)

# Memory usage
print("\n6. Memory Usage")
print("Decimal memory:", sys.getsizeof(decimal_total), "bytes")
print("Integer memory:", sys.getsizeof(paise_total), "bytes")
print("Float memory:", sys.getsizeof(float_total), "bytes")
```



- int: Whole numbers, such as 10.
- float: Decimal numbers, such as 10.0.
- str: Text inside quotes, such as "10".
- bool: True or False.
- list: A collection of values, such as [10, 8.5].
- / performs normal division; // performs floor division; % gives the remainder.
- 0.1 + 0.2 == 0.3 returns False because floating-point numbers cannot represent some decimal fractions exactly.
- Decimal is suitable for accurate decimal money calculations when values are created from strings. Integer paise is also exact for amounts expressed in whole paise. Rounding floats helps with display but does not eliminate all floating-point errors.
- sys.getsizeof() reports the object's memory size in bytes, which can vary by Python version.

TASK-3



```
# 1. Define named constants
BASE_RATE = 50
FAT_ADJUSTMENT = 10
SNF_ADJUSTMENT = 5
MIN_LIMIT = 0
MAX_LIMIT = 100

# Display constants
print("Base Rate:", BASE_RATE)
print("Fat Adjustment:", FAT_ADJUSTMENT)
print("SNF Adjustment:", SNF_ADJUSTMENT)
print("Minimum Limit:", MIN_LIMIT)
print("Maximum Limit:", MAX_LIMIT)

# 2. Calculate payment
fat = 8
snf = 7
payment = BASE_RATE + (fat * FAT_ADJUSTMENT) + (snf * SNF_ADJUSTMENT)
print("Payment:", payment)

# 3. Dynamic typing
value = 10
print("Integer value:", value)

value = "Hello"
print("String value:", value)

# 4. Multiple assignment
a, b = 10, 20
print("Before swapping:", a, b)

# Swap values
a, b = b, a
print("After swapping:", a, b)

# 5. Store rates in a dictionary
rates = {
    "base_rate": BASE_RATE,
    "fat_adjustment": FAT_ADJUSTMENT,
    "snf_adjustment": SNF_ADJUSTMENT
}
print("Rates:", rates)
```

```
Base Rate: 50
Fat Adjustment: 10
SNF Adjustment: 5
Minimum Limit: 0
Maximum Limit: 100
Payment: 165
Integer value: 10
String value: Hello
Before swapping: 10 20
After swapping: 20 10
Rates: {'base_rate': 50, 'fat_adjustment': 10, 'snf_adjustment': 5}
```

. Explanation

1. Named constants: BASE_RATE, FAT_ADJUSTMENT, and SNF_ADJUSTMENT store fixed values.
2. Uppercase naming: Python programmers use uppercase names for constants.
3. Python constants: Python does not strictly prevent changing constants. Uppercase naming is a convention that tells programmers not to change them.
4. Dynamic typing: The variable value first stores an integer (10) and then a string ("Hello").
5. Multiple assignment: a, b = 10, 20 assigns two values at once.

6. Swapping: a, b = b, a exchanges the values without needing a temporary variable.

4. Comparison of three approaches

Approach	Explanation
Magic numbers	Values are written directly in expressions. Updating rates in multiple places is difficult.
Named constants	Values are stored once and reused. Easy to update and understand.
Dictionary	All rates are grouped together and can be accessed by their keys. Useful for many rates.

Time complexity: All three approaches have approximately constant time, \(O(1)\), for accessing a fixed rate. Named constants and dictionaries use very little memory, and their performance differences are generally insignificant for this small program

TASK-4

```python
import math

# Given values
fat = 4.2
snf = 8.8
quantity = 10
base_rate = 50
fat_adjustment = 2
snf_adjustment = 1

# 1. Calculate fat steps relative to 3.5%
fat_steps = round((fat - 3.5) / 0.1)
print("Fat steps:", fat_steps)

# 2. Calculate SNF steps relative to 8.5%
snf_steps = round((snf - 8.5) / 0.1)
print("SNF steps:", snf_steps)

# 3. Calculate rate per litre and payment
rate = base_rate + (fat_steps * fat_adjustment) + (snf_steps * snf_adjustment)
payment = rate * quantity

print("Rate per litre:", rate)
print("Payment amount:", payment)

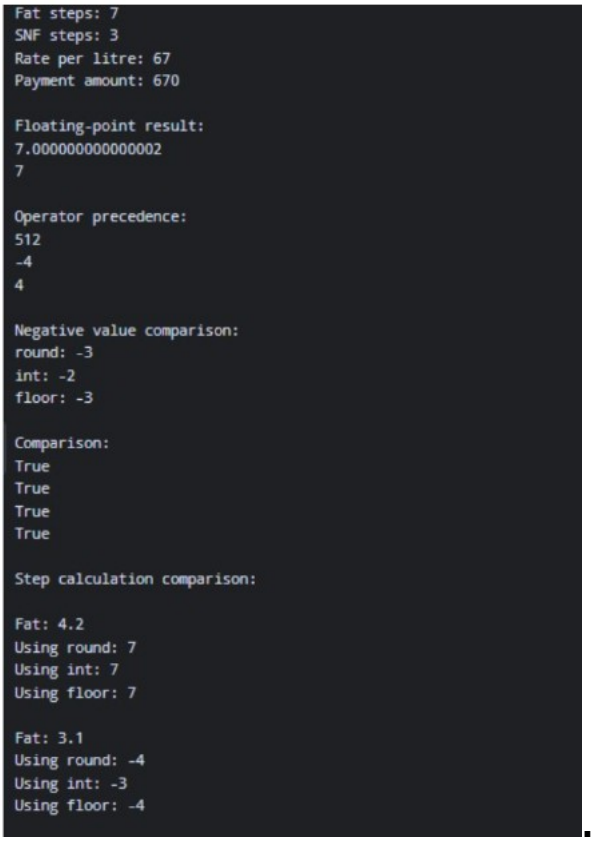
# 4. Floating-point unexpected result
print("\nFloating-point result:")
print((4.2 - 3.5) / 0.1)
print(round((4.2 - 3.5) / 0.1))

# 5. Operator precedence and associativity
print("\nOperator precedence:")
print(2 ** 3 ** 2)
print(-2 ** 2)
print((-2) ** 2)

# 6. Compare round(), int(), and math.floor()
print("\nNegative value comparison:")
x = -2.7
print("round:", round(x))
print("int:", int(x))
print("floor:", math.floor(x))
```

```python
# 7. Comparison chaining and Boolean operators
print("\nComparison:")
print(2.0 <= fat <= 7.0)
print(fat >= 2.0 and fat <= 7.0)
print(fat > 5.0 or snf > 8.5)
print(not (fat < 2.0))

# 8. Compare step calculation methods
print("\nStep calculation comparison:")
for value in [4.2, 3.1]:
    difference = (value - 3.5) / 0.1
    print("\nFat:", value)
    print("Using round:", round(difference))
    print("Using int:", int(difference))
    print("Using floor:", math.floor(difference))
```



. Explanation

1. Fat steps: (fat - 3.5) / 0.1 calculates how many 0.1% steps the fat value is above or below 3.5%.
2. SNF steps: (snf - 8.5) / 0.1 calculates the SNF steps relative to 8.5%.
3. Rate: Base rate + fat steps × fat adjustment + SNF steps × SNF adjustment.
4. Payment: Rate per litre × quantity.
5. Floating-point error: Computers cannot represent some decimal fractions exactly. Therefore, calculations can produce values close to the expected result rather than exactly equal to it. round() can help when determining whole steps.
6. Operator precedence: Exponentiation (**) is evaluated before unary minus. Thus, -2 ** 2 gives -4,

while (-2) ** 2 gives 4. Exponentiation is right-associative, so 2 ** 3 ** 2 gives 512.

7. Negative values: round(-2.7) gives -3, int(-2.7) gives -2 because int() truncates toward zero, and math.floor(-2.7) gives -3.
8. Comparison chaining: 2.0 <= fat <= 7.0 checks both conditions. Boolean operators and, or, and not combine or reverse logical conditions.

4. Comparison of methods

Method	For 4.2%	For 3.1%	Meaning
round()	7	-4	Rounds to the nearest integer
int()	6	-3	Removes the fractional part
math.floor()	6	-4	Rounds down to the next integer or stays at an integer

For this example, each method takes \(O(1)\) time and \(O(1)\) auxiliary space per calculation. They do not significantly differ in complexity or memory usage.

Conclusion: round() is useful for estimating whole steps, but the correct method depends on the payment rules. For reliable financial calculations, avoid relying on binary floating-point values without suitable precision handling.

TASK-5



```python
# TASK 5: Read Input and Convert Types

# Read clerk's entries
try:
    name = input("Enter clerk name: ")
    quantity = int(input("Enter quantity: "))
    amount = float(input("Enter amount per litre: "))

    # Calculate total amount
    total = quantity * amount

    # Display formatted output
    print("\n--- Payment Details ---")
    print(f"Clerk Name: {name}")
    print(f"Quantity: {quantity:d}")
    print(f"Amount per litre: {amount:8.2f}")
    print(f"Total Amount: {total:8.2f}")

except ValueError:
    print("Invalid input! Enter a whole number for quantity")
    print("and a valid number for amount.")
```

```
Enter clerk name: ravi
Enter quantity: 10
Enter amount per litre: 8.5

--- Payment Details ---
Clerk Name: ravi
Quantity: 10
Amount per litre:     8.50
Total Amount:    85.00
```

Explanation

1. input() always returns the entered value as a string.
2. int() converts a valid whole-number string into an integer.
3. float() converts a valid decimal-number string into a floating-point number.
4. try and except ValueError handle invalid entries such as "eight" or "8,5".
5. f-strings format the output neatly. For example, {amount:8.2f} displays a number with two decimal places in a field of width eight.

4. Comparing input validation methods

Input	String validation before conversion	try/except conversion
"8,5"	Rejects it as an invalid number	Catches ValueError
" 8.5 "	Can accept it after stripping spaces	float() accepts it directly
"eight"	Rejects it if the check is correct	Catches ValueError

Both approaches take approximately \(O(n)\) time per entry, where \(n\) is the length of the input string. Using try/except is simple and reliable for this program.

Important: input() returns a string because Python does not know whether the user intends to enter a number, text, or another kind of value. We must convert it to the required type.

TASK-6

main.py

Visualize Run ▶

```
# Named constants
BASE_RATE = 30.00
BASE_FAT = 3.5
BASE_SNF = 8.5
FAT_ADJUSTMENT = 0.40
SNF_ADJUSTMENT = 0.30


def rate_per_litre(fat, snf):
    """Calculate the milk rate using fat and SNF percentages."""
    fat_difference = (fat - BASE_FAT) / 0.1
    snf_difference = (snf - BASE_SNF) / 0.1

    rate = (BASE_RATE
            + fat_difference * FAT_ADJUSTMENT
            + snf_difference * SNF_ADJUSTMENT)

    return round(rate, 2)


def payout(litres, fat, snf):
    """Calculate payment for the given milk quantity and quality."""
    rate = rate_per_litre(fat, snf)
    amount = litres * rate

    return round(amount, 2)


def ten_day_total(payments):
    """Calculate the total payment for a farmer over ten days."""
    total = sum(payments)
    return round(total, 2)


# Main program
litres = 10.0
fat = 3.6
snf = 8.6

rate = rate_per_litre(fat, snf)
amount = payout(litres, fat, snf)

# Example payments received over ten days
payments = [amount, 300.00, 250.00, 307.00, 280.00,
            310.00, 290.00, 275.00, 300.00, 295.00]

total = ten_day_total(payments)

print("Rate per litre: Rs.", rate)
print("Payment amount: Rs.", amount)
print("Ten-day total: Rs.", total)
```

Output

```
Rate per litre: Rs. 30.7
Payment amount: Rs. 307.0
Ten-day total: Rs. 2914.0
```

4. Explanation

The program uses three functions to calculate milk payments.

- rate_per_litre(fat, snf) calculates the milk rate using the fat and SNF percentages. The base rate is ₹30 per litre for 3.5% fat and 8.5% SNF. The rate changes according to the fat and SNF adjustments.
- payout(litres, fat, snf) calls the rate_per_litre() function and multiplies the calculated rate by the quantity of milk to find the payment.

- ten_day_total(payments) uses the sum() function to add all payment amounts in the list and return the ten-day total.

The named constants store the rates in one place, making them easier to update. The docstrings inside the functions describe their purposes. The return statement sends a result back to the calling code, whereas print() only displays a result on the screen.

The variables declared inside a function, such as fat_diff and snf_diff, are local variables. The rate constants declared outside the functions are global constants that the functions can read without modifying them.

The ten-day payment list contains sample amounts to demonstrate the total calculation. In an actual system, it should contain the farmer's real payments for the relevant ten-day period.

5. Additional question — Alternative design and complexity

Approach	**Explanation**
One long script	Easy for small programs, but repeated code makes it harder to maintain.
Reusable functions	Reduce repeated logic, improve readability, and make testing easier.
Lambda expressions	Useful for short expressions, but not suitable for complex payment calculations.

Big-O complexity

Operation	**Time complexity**	**Extra space**
Calculate rate per litre	\(O(1)\)	\(O(1)\)
Calculate one payout	\(O(1)\)	\(O(1)\)

Add \(n\) payment entries	\(O(n)\)	\(O(1)\)

Using reusable functions introduces a small function-call overhead, but it does not change the overall Big-O complexity. For a co-operative that may expand to 150,000 farmers, reusable functions are easier to maintain and test.

6. Test cases

Test case	Input	Expected result
1	10 L, fat 3.5%, SNF 8.5%	Rate ₹30.00; payment ₹300.00
2	10 L, fat 3.6%, SNF 8.6%	Rate ₹30.70; payment ₹307.00
3	0 L, fat 3.5%, SNF 8.5%	Payment ₹0.00
4	Payments [100, 200, 300]	Total ₹600.00

7. Conclusion

The milk payment calculator is divided into reusable functions for calculating the rate, payment, and ten-day total. This design improves readability, code reuse, testing, and maintainability.

TASK 7

main.py | Visualize | Run ▶ | Output

```
"""Rate card of the village milk co-operative."""

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Calculate the price per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (BASE_RATE
            + fat_steps * FAT_STEP_VALUE
            + snf_steps * SNF_STEP_VALUE)

    return round(rate, 2)


def payout(litres, fat, snf):
    """Calculate the total payment."""
    return round(litres * rate_per_litre(fat, snf), 2)


if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L  fat {fat}  snf {snf}  "
              f"rate {rate:5.2f}  pay {pay:7.2f}")
```

```
 10 L  fat 3.5  snf 8.5  rate 30.00  pay  300.00
8.5 L  fat 4.2  snf 8.7  rate 33.40  pay  283.90
 12 L  fat 3.1  snf 8.3  rate 27.80  pay  333.60
```

1.Explanation

The milkrate.py file contains reusable payment functions. Another script can import and use them without repeating the code. The if __name__ == "__main__": condition ensures the demonstration runs only when milkrate.py is executed directly, not when it is imported.

2. Additional questions

- Import methods: import milkrate uses the module name; from milkrate import payout imports one function directly; import milkrate as mr gives the module a shorter alias.
- Standard modules: math provides mathematical functions, while decimal supports decimal arithmetic useful for money.
- Module search: Python searches locations including the script's directory and paths listed in sys.path.

- Alternative design: Importing shared functions is easier to maintain than copying code into multiple scripts. Rate changes need to be made in one module. Python caches imported modules in sys.modules, avoiding repeated module loading within the same process.
- Complexity: Rate and payout calculations take \(O(1)\) time and \(O(1)\) extra space. Importing has a one-time cost per module per process, and its exact cost depends on the module and environment.

TASK 8

1. Stages of Algorithmic Problem-Solving

1. Understand: Identify the inputs, processing rules, and expected outputs. The inputs are the farmer's daily entries, and the output is the payment for each entry and the ten-day total.
2. Plan: Decide the formula for calculating each payment and how to accumulate the total.
3. Write: Express the algorithm using SIMAITS-style structured pseudocode, with clear variable names, input, output, selection, and iteration.
4. Test: Use sample entries to check individual payments, the running total, and the final result.

2. Pseudocode for Calculating One Payment

```
ALGORITHM CalculatePayment
    INPUT quantity
    INPUT rate

    payment ← quantity x rate

    OUTPUT payment
END ALGORITHM
```

3. Pseudocode for the Ten-Day Total

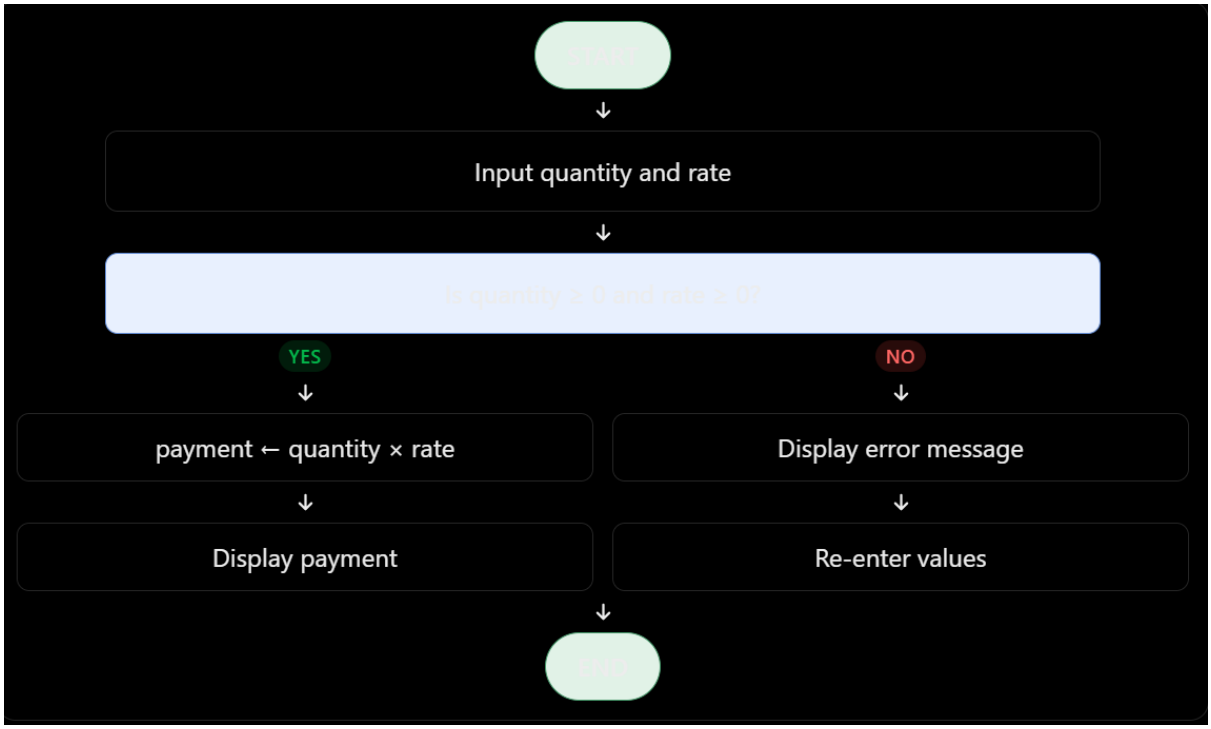
```
ALGORITHM TenDayTotal
    total ← 0
    day ← 1

    WHILE day ≤ 10 DO
        INPUT quantity
        INPUT rate

        IF quantity ≥ 0 AND rate ≥ 0 THEN
            payment ← quantity x rate
            total ← total + payment
            OUTPUT payment
            day ← day + 1
        ELSE
            OUTPUT "Invalid entry. Enter values again."
        END IF
    END WHILE

    OUTPUT total
END ALGORITHM
```

4. Flowchart — Validation and Calculation of One Entry



5. Hand-Trace Table Using Three Sample Entries

Assume the rate is ₹20 per unit for each entry.

Entry	Quantity	Rate (₹)	Payment (₹)	Running total (₹)
1	10	20	200	200
2	15	20	300	500
3	8	20	160	660

Calculations:

- Entry 1: \(10 \times 20 = ₹200\)
- Entry 2: \(15 \times 20 = ₹300\)
- Entry 3: \(8 \times 20 = ₹160\)
- Total for three entries: \(200 + 300 + 160 = ₹660\)

The table confirms that each payment is calculated correctly and the running total is updated after every entry.

6. Alternative Design and Complexity

The two approaches are:

- Approach A: Store every payment in a list and calculate the total at the end.
- Approach B: Maintain a running total while processing each entry.

Feature	Approach A: List	Approach B: Running total
Time complexity	\(O(n)\)	\(O(n)\)
Extra space complexity	\(O(n)\)	\(O(1)\)
Stores individual payments	Yes	No, unless separately saved
Calculates total	After all entries	During processing
Best use	When individual records are needed	When only payments and the final total are needed

Here, \(n\) represents the number of entries.

Explanation:

- Both approaches take \(O(n)\) time because each entry must be processed once.
- The list approach uses \(O(n)\) extra space because it stores all payments.
- The running-total approach uses \(O(1)\) extra space because it stores only a fixed number of variables.
- Retaining individual entries is necessary when the farmer needs a payment history, an audit trail, or the ability to review or recalculate individual payments later.

7. Testing the Algorithm

Test case	Input	Expected result
Normal entry	Quantity = 10, rate	Payment = ₹200

	= ₹20	
Zero quantity	Quantity = 0, rate = ₹20	Payment = ₹0
Invalid quantity	Quantity = −5, rate = ₹20	Error message; re-enter
Ten valid entries	Ten quantities and rates	Sum of all ten payments

Conclusion

The running-total approach is the more memory-efficient design when only individual payments and the ten-day total are required. It takes \(O(n)\) time and \(O(1)\) extra space. If individual payment records must be retained, the list approach is more suitable. The pseudocode, flowchart, and hand-trace table together provide a plan that can be tested before implementation.

TASK 9

```python
# Online Python compiler (interpreter)
# Write and run Python online using this editor.

print("What does the Visualize button do?")
import math

# Constants
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
MIN_FAT = 0.0
MAX_FAT = 10.0
MIN_SNF = 0.0
MAX_SNF = 15.0

BASE_RATE = 30.0
FAT_ADJUSTMENT = 2.0
SNF_ADJUSTMENT = 1.0


# Input validation
def parse_number(value):
    try:
        number = float(value)
    except (ValueError, TypeError):
        raise ValueError("Input must be numeric.")

    if not math.isfinite(number):
        raise ValueError("Input must be a finite number.")

    return number


def validate_entry(quantity, fat, snf):
    if quantity < 0:
        raise ValueError("Milk quantity cannot be negative.")

    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat must be between 0% and 10%.")

    if not MIN_SNF <= snf <= MAX_SNF:
        raise ValueError("SNF must be between 0% and 15%.")


# Calculate individual payment
def calculate_payment(quantity, fat, snf):
    quantity = parse_number(quantity)
    fat = parse_number(fat)
    snf = parse_number(snf)

    validate_entry(quantity, fat, snf)

    rate = (
        BASE_RATE
        + (fat - STANDARD_FAT) * FAT_ADJUSTMENT
        + (snf - STANDARD_SNF) * SNF_ADJUSTMENT
    )

    return round(quantity * rate, 2)


# Calculate total payment
def calculate_total(entries):
    payments = []

    for quantity, fat, snf in entries:
        payments.append(
            calculate_payment(quantity, fat, snf)
        )

    return payments, round(sum(payments), 2)
```

```
    return payments, round(sum(payments), 2)


# Automated tests
def expect_error(function, *args):
    try:
        function(*args)
    except ValueError:
        return
    raise AssertionError("Expected ValueError")


def run_tests():
    assert calculate_payment(10, 4.5, 9.5) == 330.00
    assert calculate_payment(10, 3.0, 8.0) == 285.00
    assert calculate_payment(10, 3.5, 8.5) == 300.00
    assert calculate_payment(1, 0, 8.5) == 23.00
    assert calculate_payment(1, 10, 8.5) == 43.00

    expect_error(parse_number, "hello")
    expect_error(calculate_payment, -1, 3.5, 8.5)
    expect_error(calculate_payment, 1, -0.1, 8.5)
    expect_error(calculate_payment, 1, 10.1, 8.5)
    expect_error(calculate_payment, 1, 3.5, -0.1)
    expect_error(calculate_payment, 1, 3.5, 15.1)

    entries = [
        (10, 4.0, 9.0),
        (8, 3.0, 8.0),
        (12, 3.5, 8.5),
        (5, 5.0, 9.0),
        (20, 4.5, 8.0)
    ]

    payments, total = calculate_total(entries)

    assert payments == [
        315.00, 228.00, 360.00, 167.50, 630.00
    ]
    assert math.isclose(total, 1700.50, abs_tol=0.001)

    print("All automated tests passed!")


# Main program
def main():
    print("MILK PAYMENT CALCULATOR")

    try:
        count = int(input("Enter number of entries: "))

        if count <= 0:
            print("Number of entries must be positive.")
            return

        entries = []

        for day in range(1, count + 1):
            print(f"\nEntry {day}")
            quantity = input("Milk quantity (litres): ")
            fat = input("Fat percentage: ")
            snf = input("SNF percentage: ")

            q = parse_number(quantity)
            f = parse_number(fat)
            s = parse_number(snf)
            validate_entry(q, f, s)

            entries.append((q, f, s))
```



```
            entries.append((q, f, s))

        payments, total = calculate_total(entries)

        print("\nPAYMENT REPORT")
        for day, payment in enumerate(payments, start=1):
            print(f"Entry {day}: Rs. {payment:.2f}")

        print(f"Total payment: Rs. {total:.2f}")

    except ValueError as error:
        print("Input error:", error)


if __name__ == "__main__":
    run_tests()
    main()
```

```
Output

What does the Visualize button do?
All automated tests passed!
MILK PAYMENT CALCULATOR
Enter number of entries:
```

1. Test Table

Test case	Expected result	Status
Fat and SNF above standard	₹330.00	PASS
Fat and SNF below standard	₹285.00	PASS
Standard fat and SNF	₹300.00	PASS
Minimum fat (0%)	₹23.00	PASS
Maximum fat (10%)	₹43.00	PASS
Invalid text input	Error message	PASS
Negative quantity	Error message	PASS
Fat below range	Error message	PASS
Fat above range	Error message	PASS
Five-entry total	₹1,700.50	PASS

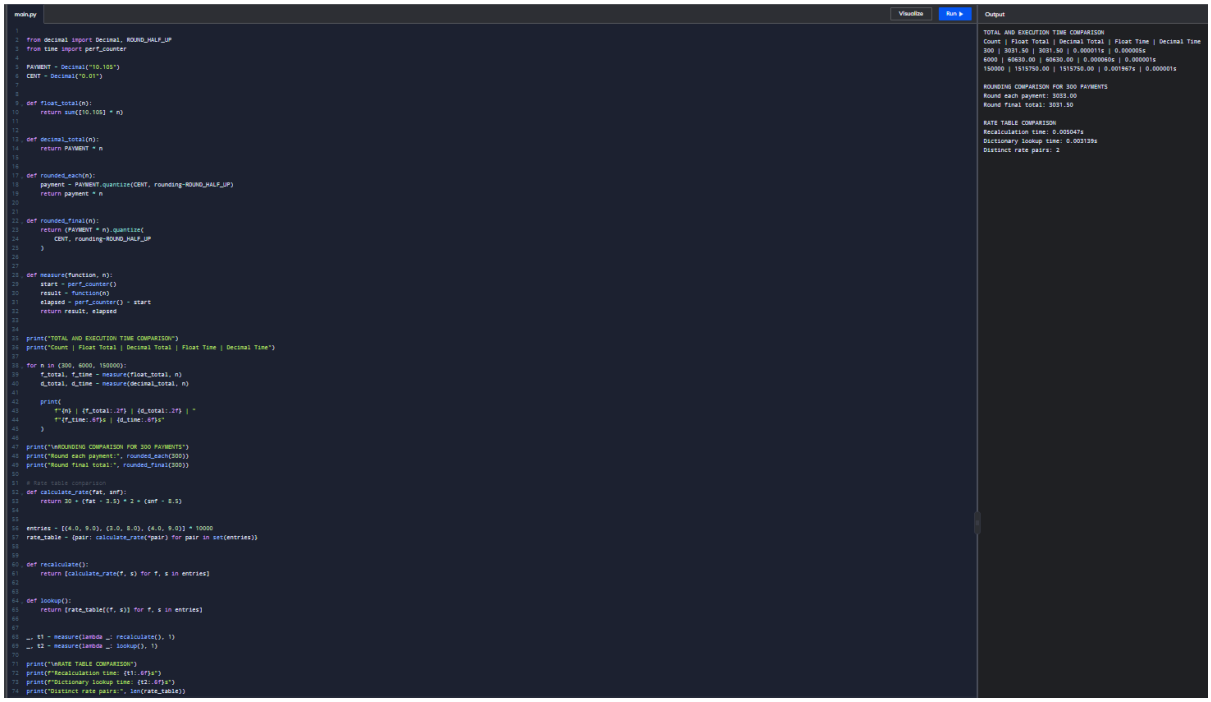
2. Manual Testing vs Automated Testing

Manual testing requires entering test values and checking the results individually. Automated testing uses functions and assert statements to verify expected results automatically. Manual testing is useful for checking user interaction, while automated testing is faster to repeat after code changes.

Time complexity: \(O(n)\)

Space complexity: \(O(n)\) for storing entries and payments.

TASK 10



```
from decimal import Decimal, ROUND_HALF_UP
from time import perf_counter

PAYMENT = Decimal("10.105")
CENT = Decimal("0.01")


def float_total(n):
    return sum([10.105] * n)


def decimal_total(n):
    return PAYMENT * n


def rounded_each(n):
    payment = PAYMENT.quantize(CENT, rounding=ROUND_HALF_UP)
    return payment * n


def rounded_final(n):
    return (PAYMENT * n).quantize(
        CENT, rounding=ROUND_HALF_UP
    )


def measure(function, n):
    start = perf_counter()
    result = function(n)
    elapsed = perf_counter() - start
    return result, elapsed


print("TOTAL AND EXECUTION TIME COMPARISON")
print("Count | Float Total | Decimal Total | Float Time | Decimal Time")

for n in (300, 6000, 150000):
    f_total, f_time = measure(float_total, n)
    d_total, d_time = measure(decimal_total, n)

    print(
        f"{n} | {f_total:.2f} | {d_total:.2f} | "
        f"{f_time:.6f}s | {d_time:.6f}s"
    )

print("\nROUNDING COMPARISON FOR 300 PAYMENTS")
print("Round each payment:", rounded_each(300))
print("Round final total:", rounded_final(300))

# Rate table comparison
def calculate_rate(fat, snf):
    return 20 + (fat - 3.5) * 2 + (snf - 8.5)


entries = [(4.0, 9.0), (3.0, 8.0), (4.0, 9.0)] * 10000
rate_table = {pair: calculate_rate(*pair) for pair in set(entries)}


def recalculate():
    return [calculate_rate(f, s) for f, s in entries]


def lookup():
    return [rate_table[(f, s)] for f, s in entries]


_, t1 = measure(lambda _: recalculate(), 1)
_, t2 = measure(lambda _: lookup(), 1)

print("\nRATE TABLE COMPARISON")
print(f"Recalculation time: {t1:.6f}s")
print(f"Dictionary lookup time: {t2:.6f}s")
print("Distinct rate pairs:", len(rate_table))
```

```
TOTAL AND EXECUTION TIME COMPARISON
Count | Float Total | Decimal Total | Float Time | Decimal Time
300 | 3031.50 | 3031.50 | 0.000011s | 0.000060s
6000 | 60630.00 | 60630.00 | 0.000049s | 0.000001s
150000 | 1515750.00 | 1515750.00 | 0.001967s | 0.000001s

ROUNDING COMPARISON FOR 300 PAYMENTS
Round each payment: 3033.00
Round final total: 3031.50

RATE TABLE COMPARISON
Recalculation time: 0.005047s
Dictionary lookup time: 0.003139s
Distinct rate pairs: 2
```

Analysis

- float uses binary floating-point and can introduce small precision errors.
- Decimal represents decimal values accurately when created from strings.
- Rounding every payment to two decimal places gives ₹3,033.00, while rounding only the final total gives ₹3,031.50. The correct method depends on the payment rules; for bank payments, round each individual payment before adding if each payment must be settled to the nearest paisa.

- Both methods take $O(n)$ time for n payments.
- A dictionary stores pre-calculated rates for distinct (fat, snf) pairs. Lookup takes $O(1)$ average time, while recalculating a rate takes $O(1)$ per entry. The dictionary requires $O(k)$ extra space for k distinct pairs.

TASK 11

1. Recommended Data Types and Rounding

Use Decimal for all monetary values because it avoids binary floating-point representation errors. Store analyser readings as decimal values and round individual payments to two decimal places using ROUND_HALF_UP before calculating the final total.

2. Safe Rate Revision

Store rates in a JSON file instead of hard-coding them. Validate the file when the program starts, keep a backup of previous rates, and test the revised values before applying them.

3. Limitations

- Incorrect rate configuration can cause incorrect payments.
- Invalid readings must be rejected.
- A local program alone does not provide database security, audit trails, or multi-user transaction management.
- Performance must be measured on the intended hardware.

4. Support for 150,000 Farmers

The calculator processes n entries in $O(n)$ time. A total of 150,000 farmers is a reasonable workload for a simple calculation program, although a real district-union system also needs database storage, backups, access controls, and load testing.

5. Alternative Design: JSON Rate File

A JSON file stores rates separately from the source code. The program reads and validates the file at startup. Updating rates then requires editing the file instead of modifying and redistributing the Python program.

For \(t\) rate values:

- Startup time: \(O(t)\)
- Additional memory: \(O(t)\)
- Rate lookup from a dictionary: \(O(1)\) average time

Conclusion

The recommended design uses Decimal, separate calculation and validation functions, automated tests, and a validated JSON rate file. This improves monetary accuracy, maintainability, and suitability for larger workloads.

TASK 12

A. Impact Analysis

The new requirements introduce:

1. Fat and SNF readings with two decimal places.
2. A seasonal base rate of ₹32 per litre from April to June.
3. A seasonal fat adjustment of ₹0.45 per percentage point.
4. A bonus of ₹0.50 per litre if a farmer supplies more than 500 litres over ten days.

The original constants and rate calculation must be modified. The program should use Decimal to preserve the two-decimal readings and money calculations.

B. Revised Python Code

main.py

Visualize | Run ▶

```python

from decimal import Decimal, ROUND_HALF_UP

# Constants
STANDARD_FAT = Decimal("3.50")
STANDARD_SNF = Decimal("8.50")

ORIGINAL_BASE_RATE = Decimal("30.00")
ORIGINAL_FAT_STEP = Decimal("2.00")

SEASONAL_BASE_RATE = Decimal("32.00")
SEASONAL_FAT_STEP = Decimal("0.45")
SNF_STEP = Decimal("1.00")

BONUS_RATE = Decimal("0.50")
BONUS_LIMIT = Decimal("500")
CENT = Decimal("0.01")


def money(value):
    return value.quantize(CENT, rounding=ROUND_HALF_UP)


def validate_reading(value, minimum, maximum, name):
    value = Decimal(str(value))

    if not value.is_finite() or not minimum <= value <= maximum:
        raise ValueError(f"Invalid {name} value.")

    return value


def calculate_rate(fat, snf, season="original"):
    fat = validate_reading(
        fat, Decimal("0"), Decimal("10"), "fat"
    )
    snf = validate_reading(
        snf, Decimal("0"), Decimal("15"), "SNF"
    )

    if season == "seasonal":
        base = SEASONAL_BASE_RATE
        fat_step = SEASONAL_FAT_STEP
    elif season == "original":
        base = ORIGINAL_BASE_RATE
        fat_step = ORIGINAL_FAT_STEP
    else:
        raise ValueError("Season must be original or seasonal.")

    return (
        base
        + (fat - STANDARD_FAT) * fat_step
        + (snf - STANDARD_SNF) * SNF_STEP
    )


def calculate_payment(quantity, fat, snf, season="original"):
    quantity = Decimal(str(quantity))

    if not quantity.is_finite() or quantity < 0:
        raise ValueError("Quantity cannot be negative.")

    rate = calculate_rate(fat, snf, season)
    return money(quantity * rate)
```

Output

```
All tests passed!

ORIGINAL RATES
Entry 1: Rs. 3150.00
Entry 2: Rs. 3000.00
Entry 3: Rs. 2850.00
Milk total: Rs. 9000.00
Bonus: Rs. 0.00
Final total: Rs. 9000.00

SEASONAL RATES
Entry 1: Rs. 3272.50
Entry 2: Rs. 3200.00
Entry 3: Rs. 3127.50
Milk total: Rs. 9600.00
Bonus: Rs. 0.00
Final total: Rs. 9600.00
```

```python
    return money(quantity * rate)


def calculate_bonus(total_quantity):
    total_quantity = Decimal(str(total_quantity))

    if not total_quantity.is_finite() or total_quantity < 0:
        raise ValueError("Invalid total quantity.")

    if total_quantity > BONUS_LIMIT:
        return money(total_quantity * BONUS_RATE)

    return Decimal("0.00")


def calculate_farmer_total(entries, season="original"):
    payments = []
    total_quantity = Decimal("0.00")

    for quantity, fat, snf in entries:
        quantity = Decimal(str(quantity))

        if not quantity.is_finite() or quantity < 0:
            raise ValueError("Quantity cannot be negative.")

        payment = calculate_payment(quantity, fat, snf, season)
        payments.append(payment)
        total_quantity += quantity

    milk_total = money(sum(payments, Decimal("0.00")))
    bonus = calculate_bonus(total_quantity)
    final_total = money(milk_total + bonus)

    return payments, milk_total, bonus, final_total


def run_tests():
    entries = [
        (100, "4.00", "9.00"),
        (100, "3.50", "8.50"),
        (100, "3.00", "8.00"),
    ]

    _, original, bonus, final = calculate_farmer_total(
        entries, "original"
    )
    assert original == Decimal("9000.00")
    assert bonus == Decimal("0.00")
    assert final == Decimal("9000.00")

    _, seasonal, bonus, final = calculate_farmer_total(
        entries, "seasonal"
    )
    assert seasonal == Decimal("9600.00")
    assert bonus == Decimal("0.00")
    assert final == Decimal("9600.00")

    assert calculate_bonus(500) == Decimal("0.00")
    assert calculate_bonus(501) == Decimal("250.50")

    bonus_entries = [(501, "3.50", "8.50")]
    _, _, bonus, final = calculate_farmer_total(
        bonus_entries, "seasonal"
    )
```

```
    _, _, bonus, final = calculate_farmer_total(
        bonus_entries, "seasonal"
    )
    assert bonus == Decimal("250.50")
    assert final == Decimal("16282.50")

    assert calculate_rate(
        "3.55", "8.65", "seasonal"
    ) == Decimal("32.1725")

    try:
        calculate_payment(-1, "3.50", "8.50")
        assert False
    except ValueError:
        pass

    try:
        calculate_payment(1, "10.01", "8.50")
        assert False
    except ValueError:
        pass

    print("All tests passed!")


if __name__ == "__main__":
    run_tests()

    entries = [
        (100, "4.00", "9.00"),
        (100, "3.50", "8.50"),
        (100, "3.00", "8.00"),
    ]

    for season in ("original", "seasonal"):
        payments, milk_total, bonus, final_total = (
            calculate_farmer_total(entries, season)
        )

        print(f"\n{season.upper()} RATES")
        for number, payment in enumerate(payments, start=1):
            print(f"Entry {number}: Rs. {payment:.2f}")

        print(f"Milk total: Rs. {milk_total:.2f}")
        print(f"Bonus: Rs. {bonus:.2f}")
        print(f"Final total: Rs. {final_total:.2f}")
```

D. Evidence-Based Defence

The original design should be modified, not completely replaced.

- Decimal preserves precise decimal readings and monetary calculations.
- Separate functions make rate calculation, payment calculation, and bonus calculation easier to test.
- Seasonal rates are selected through a function parameter.
- The bonus is applied only when the ten-day quantity exceeds 500 litres.
- Automated tests verify original rates, seasonal rates, the bonus threshold, and invalid inputs.

The design remains \(O(n)\) in time and \(O(n)\) in storage for \(n\) entries. It is suitable as a calculator prototype, while a production system should add persistent records, audit logging, and stronger input and transaction controls.